Taller 2: Construyendo el procesador ArquiSim — Respuestas a las Preguntas de Control

Pontificia Universidad Javeriana — Arquitectura de Computadores (2026-3)

Integrante: Jose Miguel Fork: https://github.com/vargasjosemiguel2007-code/ArquiSim

Nota sobre las capturas: las “capturas” de este documento son la salida de texto del simulador (modo --traza / estado final), que muestra los mismos valores que los paneles de la interfaz. Las pedidas en 2.1, 3.4, 4.2 y 5.5 deben reemplazarse o complementarse con una captura de pantalla de la interfaz interactiva si el profesor la exige.

Etapa 0

0.1

El programa 00_suma tiene 4 instrucciones y ocupa 16 bytes (0x000 a 0x00F). Cada instrucción mide 32 bits = 4 bytes y la RAM se direcciona por byte, así que la siguiente instrucción está 4 direcciones más adelante; por eso el PC avanza de 4 en 4.

0.2

add $s0, $t0, $t1 = 0x01098020 = 0000 0001 0000 1001 1000 0000 0010 0000.

```
op      rs     rt     rd     shamt  funct
000000  01000  01001  10000  00000  100000
0       8      9      16     0      0x20 (add)
```

$s0 = registro 16, $t0 = registro 8, $t1 = registro 9.

0.3

−3 en 32 bits es 0xFFFFFFFD. La suma sin signo 20 + 0xFFFFFFFD = 0x1_00000011 no cabe en 32 bits: sale un acarreo del bit 31, por eso C = 1 (la resta 20 − 3 se hace como suma del complemento a dos). El resultado 0x11 = 17 es correcto. V = 0 porque sumar un positivo y un negativo nunca desborda con signo: el resultado siempre cabe.

0.4

Con $t0 = 100: $s0 = 100 + 15 = 115 (0x73) y $s1 = 115 − 3 = 112 (0x70). El comentario del programa (20 y 17) supone $t0 = 5; los comentarios son texto, no código, y el procesador solo usa el valor actual del registro.

Etapa 1

1.1

La última instrucción es sub $s5, $t1, $t0 con $t1 = 5 y $t0 = 15 en este programa (5 − 15 = −10 = 0xFFFFFFF6; el enunciado cita 5 − 10 como ejemplo). Banderas: Z = 0 (resultado ≠ 0), N = 1 (bit 31 = 1), C = 1 (préstamo: 5 < 15 sin signo), V = 0 (5 − 15 no se sale del rango con signo).

1.2

0x8A2C9031 desplazado 28 posiciones. srl rellena con ceros → 0x00000008. sra copia el bit de signo (1) → 0xFFFFFFF8. Si andi usara in->simm: 0xFFFF se extiende con signo a 0xFFFFFFFF y la máscara no borra nada, así que $t3 = $s1 = 0x8A2C9031 en vez de 0x00009031.

1.3

```
                        $t0             $t1
inicio                  0000 1111 0000  0000 1111 1111
xor $t0,$t0,$t1         0000 0000 1111  0000 1111 1111
xor $t1,$t0,$t1         0000 0000 1111  0000 1111 0000
xor $t0,$t0,$t1         0000 1111 1111  0000 1111 0000
```

Tras xor $s3, $t0, $t0 el resultado es 0 (x XOR x = 0): se enciende Z = 1; N = 0, C = 0 y V = 0 (las lógicas ponen C = V = 0).

1.4

Mensaje: `EXCEPCIÓN en PC = 0x008: desbordamiento aritmético en addi`. Con addiu no hay excepción: $t1 = 0x80000000, que con signo es −2147483648; banderas N = 1, V = 1, C = 0, Z = 0 (0x7FFFFFFF + 1 cambia de signo, pero no hay acarreo).

1.5

0x00000400 + 0xFFFFFFFC (−4) = 0x1_000003FC: hay acarreo (C = 1) porque −4 se escribe como un número sin signo enorme. No hay desbordamiento con signo (V = 0): 1024 + (−4) = 1020 cabe en el rango, y sumar números de distinto signo nunca desborda.

Etapa 2

2.1

A empieza en 0x200 y A[12] queda en 0x200 + 48 = 0x230. El desplazamiento está en bytes, no en elementos: cada palabra mide 4 bytes, 12 × 4 = 48. Captura (m A, valor 104 = 0x68 guardado):

```
Dir     Bytes (big-endian)   Palabra          Decimal
0x230   00 00 00 68          0x00000068          104
```

2.2

Mensaje: `EXCEPCIÓN en PC = 0x008: dirección 0x206 no está alineada a 4 bytes.` Alineación: una palabra de 32 bits debe empezar en una dirección múltiplo de 4. El bus de memoria lee palabras completas en un solo acceso; una dirección no alineada cruzaría dos palabras y exigiría dos accesos y circuitos para unir los bytes, por eso el hardware la prohíbe.

2.3

Al final: 0x200 = 0x48 (H), 0x201 = 0x4F (O), 0x202 = 0x4C (L), 0x203 = 0x41 (A); la palabra es 0x484F4C41. Es big-endian porque el byte más significativo (0x48) está en la dirección más baja, de modo que los bytes leídos en orden de dirección son los dígitos de la palabra de izquierda a derecha.

2.4

0x80 = 1000 0000 tiene el bit 7 en 1: lb extiende el signo → 0xFFFFFF80 (−128); lbu extiende con ceros → 0x00000080 (128). 0xDF = 1101 1111: apaga el bit 5 (valor 0x20). Las minúsculas ASCII son las mayúsculas con ese bit en 1 (‘a’ = 0x61 = 0110 0001, ‘A’ = 0x41 = 0100 0001), así que AND con 0xDF las convierte en mayúsculas.

Etapa 3

3.1

bne $s3, $s4, ELSE = 0x16740002 → inmediato = 2. Tras el bne el PC vale 0x014 y ELSE está en 0x01C: (0x01C − 0x014) / 4 = 2 instrucciones que se saltan (el add y el j). Con j = 8: se toma el else y $s0 = 10 − 3 = 7 con 6 instrucciones ejecutadas; con j = 7: $s0 = 13 con 7 instrucciones.

3.2

El cuerpo se ejecuta 4 veces (A[3], A[2], A[1], A[0]); la condición se evalúa 5 veces. $t0 parte en 0x20C y baja 4 por vuelta; después de procesar A[0] (0x200) hace p-- una vez más y queda en 0x1FC: slt detecta p < &A[0] y sale. C = 1 la dejó el último `addi $t0, $t0, -4` (0x200 + 0xFFFFFFFC produce acarreo). slt no cuenta porque solo escribe 0 o 1 y no toca las banderas.

3.3

Los bits son los mismos (0xFFFFFFFF contra 1), pero slt los interpreta con signo (−1 < 1 → 1) y sltu sin signo (4294967295 < 1 → 0). Es la misma comparación sobre dos interpretaciones distintas del patrón de bits.

3.4

En 13_funcion, jal (en 0x004) guarda en $ra el valor 0x008: PC + 4, la dirección de la instrucción siguiente al jal, para que jr $ra vuelva justo después de la llamada. En 14_pila, con bp DOBLE (ventana PILA, $sp = 0x3FC):

```
PILA ($sp = 0x3FC)
0x3FC   00 00 00 08          0x00000008           8
```

En 0x3FC está el $ra que guardó DOBLE_MAS_UNO con sw: 0x008 (retorno a main). Al hacer el segundo jal, $ra pasa a valer 0x01C y la dirección de main solo sobrevive en la pila.

3.5

Sin sw/lw de $ra el programa no termina: se alcanza el límite de 10000 instrucciones. El segundo jal sobrescribe $ra con 0x018; DOBLE vuelve a 0x018 y, al llegar al jr $ra de DOBLE_MAS_UNO, $ra sigue valiendo 0x018, así que salta otra vez a 0x018 en un ciclo infinito (nunca vuelve a main, $v0 crece y $sp se sigue moviendo).

3.6

El programador escribió 8 líneas de código y en memoria hay 9 instrucciones. li $t1, 0x12345678 se convirtió en `lui $t1, 0x1234` + `ori $t1, $t1, 0x5678`: el campo inmediato tiene solo 16 bits, y la constante de 32 bits se reparte en dos mitades.

Etapa 4

4.1

Píxel (5, 3): 0x400 + 32·3 + 5 = 0x465. Para pintarlo de verde (probado en respuestas/pixel_5_3.asm):

```
addi $t0, $zero, 2
sb   $t0, 0x465($zero)
```

sll $t1, $t0, 5 multiplica por 2^5 = 32 (el ancho de una fila), así que calcula 32·y. El borde inferior es la fila 15: 15 × 32 = 480 bytes más allá de la fila 0.

4.2

Sondeo (polling): el programa lee el registro de botones (lw de 0x600) una y otra vez en un ciclo; el dispositivo no avisa. Si un botón se activa y se desactiva mientras el programa está dentro del ciclo BIT, la foto de los botones ya fue tomada con lw, así que ese pulso se pierde. (Captura de 17_botones en tiempo real con a 10: debe tomarse en la interfaz; aquí no se generó.)

4.3

La pantalla de 16 colores ocupa 512 direcciones (0x400–0x5FF); la RGB solo 2 (0x604 y 0x608). 18_pantalla_rgb hace 1024 escrituras de color (32 × 32) más una de posición al inicio. No necesita escribir la posición antes de cada color porque escribir el color dibuja y avanza x automáticamente (autoincremento). Al final mem[0x604] = 0: tras 1024 píxeles la posición da la vuelta a (0, 0).

Etapa 5

5.1

```
add $s1, $t2          0x022A8820
add $s1, $s1, $t2     0x022A8820   <- idéntica
add $t1               0x00090030   <- distinta (funct 0x30)
```

add $s1, $t2 y add $s1, $s1, $t2 tienen el mismo código máquina; add $t1 es otra instrucción (otro funct, usa ACC). Una instrucción de 2 direcciones es una sintaxis nueva de la misma instrucción, no una instrucción nueva.

5.2

01_aritmetica ejecuta 8 instrucciones y usa 8 registros del banco ($s0–$s5, $t0, $t1). 20_acumulador ejecuta 10 y usa 5 ($s0–$s4; el resto pasa por ACC). Ventaja: instrucciones más simples (un operando) y menos registros. Desventaja: todo pasa por un único registro, hay más instrucciones y se pierden los resultados intermedios.

5.3

(a) última sub $t1 del ciclo (ACC 1 → 0): Z = 1, N = 0, C = 0, V = 0. (b) cmp $t2, $t3 (3 − 7): Z = 0, N = 1, C = 1, V = 0. (c) addi 1 sobre ACC = 0x7FFFFFFF: Z = 0, N = 1, C = 0, V = 1. Una instrucción entre cmp y el salto rompe la decisión: tras cmp 3 − 7 queda N = 1, pero si se intercala addi $s1, $zero, 1 (resultado positivo), N pasa a 0 y jn ya no salta aunque 3 < 7.

5.4

mult $t0, $t0 con 100000: HI = 2, LO = 0x540BE400. 10^10 = HI · 2^32 + LO = 2 · 4294967296 + 1410065408. ble $t6, 255, LISTO se convirtió en:

```
addi $at, $zero, 255
slt  $at, $at, $t6
beq  $at, $zero, LISTO
```

$at está reservado porque el ensamblador necesita un registro temporal para expandir pseudoinstrucciones sin destruir ningún registro del programador.

5.5

```
Ingrese un número (0 para terminar): 3
  su cuadrado es 9
Ingrese un número (0 para terminar): 4
  su cuadrado es 16
Ingrese un número (0 para terminar): 0
Suma de los cuadrados: 25
```

addi $s7, $zero, 1 nunca se ejecuta porque antes está el syscall con $v0 = 10, que termina el programa.

Reto final

programas/24_reto.asm y 24_reto.esperado: arreglo de 8 palabras con 4 negativos, función CONTAR (jal/jr) que devuelve la cantidad en $v0, resultado guardado en NEG (0x220) y 4 píxeles rojos en la fila 0. make test: 25 de 25 programas pasan.